

Fine-Grained and Lightweight Quantum-Resistant Access Control System With Efficient Revocation for IoT Cloud

POTCHAKORN POOMEKUM^{ID}, ARISARA SURIYAWONG, AND SOMCHART FUGKEAW^{ID} (Member, IEEE)

School of ICT, Sirindhorn International Institute of Technology, Thammasat University, Pathum Thani 12121, Thailand

CORRESPONDING AUTHOR: S. FUGKEAW (e-mail: somchart@siit.tu.ac.th)

ABSTRACT The proliferation of Internet of Things (IoT) devices in cloud-assisted environments raises urgent concerns regarding fine-grained access control, policy privacy, and resilience against quantum adversaries. To address these challenges, we propose a lightweight and quantum-resistant access control framework that combines Ring-LWE-based Ciphertext-Policy Attribute-Based Encryption (CP-ABE) with a fog-assisted split-path partial encryption architecture. In our scheme, IoT devices encrypt raw data using AES-256-GCM, while fog nodes perform the RLWE-CP-ABE encapsulation of session keys, thereby reducing device-side computation while ensuring that neither plaintext nor session keys are exposed to semi-trusted fog nodes. To preserve policy confidentiality, we integrate a salted attribute-hashing mechanism into the Linear Secret Sharing Scheme (LSSS) matrices, concealing access policies even during fog-side processing. Furthermore, we design an epoch-based and blacklist-based revocation mechanism that supports both user-level and attribute-level revocation without requiring ciphertext re-encryption. Security is guaranteed through CRYSTALS-Dilithium signatures and Poly1305 message authentication, which provide post-quantum authenticity and integrity. Experimental results confirm that our framework significantly reduces computational and communication overhead while achieving scalable, verifiable, and post-quantum-secure access control—outperforming existing lattice-based CP-ABE schemes in both efficiency and security.

INDEX TERMS IoT cloud, CP-ABE, post-quantum cryptography, lattice-based cryptography, RLWE, fog computing, access control, CRYSTALS-Dilithium.

I. INTRODUCTION

THE INTERNET of Things (IoT) is fundamentally transforming critical infrastructures by enabling pervasive connectivity and autonomous data exchange among billions of devices across sectors such as healthcare, transportation, smart cities, and industrial control systems. These interconnected devices continuously generate vast volumes of sensitive data, which is offloaded to distributed fog and cloud infrastructures for scalable storage and real-time analytics.

However, securing this data in large-scale, multi-tenant environments remains a significant challenge. This problem is intensified by the emerging threat of quantum computing.

Conventional access control mechanisms in IoT systems predominantly rely on classical cryptographic primitives,

including RSA, ECC, and Ciphertext-Policy Attribute-Based Encryption (CP-ABE). Despite their widespread use, these schemes are vulnerable to quantum adversaries due to the underlying hardness assumptions being susceptible to Shor's and Grover's algorithms. In contrast, post-quantum cryptography (PQC), particularly schemes grounded in the Learning With Errors (LWE) and Ring-LWE problems [1], [2], [3], has emerged as a promising alternative. Among them, lattice-based CP-ABE provides a powerful foundation for fine-grained access control by embedding attribute policies directly into ciphertexts, ensuring that only authorized users can decrypt the data.

Nonetheless, existing lattice-based CP-ABE schemes [4], [5], [6], [7], [8], [9], [10], [30] are still not suitable for deployment in resource-constrained IoT-cloud environments due to several unresolved problems:

- 1) *Excessive Computational and Communication Overhead*: High-dimensional polynomial operations result in prohibitive costs for encryption, decryption, and key generation. Combined with bulky ciphertext structures, this overhead makes lattice CP-ABE impractical for IoT devices with limited computing power and for bandwidth-constrained IoT networks.
- 2) *Absence of Secure Fog Delegation*: While fog computing offers natural opportunities to offload computation, existing lattice CP-ABE frameworks lack secure mechanisms that allow fog nodes to assist with partial decryption or policy enforcement without compromising data confidentiality.
- 3) *Policy Exposure and Privacy Risks*: Many CP-ABE constructions embed access structures in plaintext, enabling adversaries or semi-trusted fog nodes to infer user attributes, which can lead to profiling attacks or leakage of sensitive access patterns.
- 4) *Inefficient and Incomplete Revocation*: Current revocation mechanisms [6], [14], [15], [17], [25], [31] often require ciphertext re-encryption or system-wide key redistribution, which are computationally expensive and disruptive. Moreover, most schemes focus only on attribute-level revocation, neglecting user-level revocation and forward secrecy.

Recent efforts have attempted to mitigate subsets of these challenges. Hybrid encryption models [11], [12], [13] leverage symmetric encryption with lattice-based key encapsulation to reduce device-side overhead but fall short in enabling fog-enforced, fine-grained access control. Similarly, revocable lattice-based CP-ABE constructions [6], [14], [15] introduce mediator-based revocation, yet lack support for dynamic policy hiding and real-time delegation. Post-quantum authentication solutions [16], [17] address data integrity but overlook policy enforcement and access coordination. Critically, no existing framework provides a unified solution that integrates quantum-resilient encryption, policy privacy, fine-grained and forward-secure revocation, and fog-assisted secure access enforcement.

To address these gaps, we propose a fog-assisted, quantum-resilient, and revocation-aware access control framework tailored for secure IoT data sharing. To the best of our knowledge, our paper is the first to provide a unified solution for secure and fine-grained lattice CP-ABE access control in Fog-assisted IoT cloud. The proposed system introduces the following core innovations:

- 1) *Split-Path Partial Encryption with Fog-Assisted Encapsulation*: We introduce an architectural refinement in which IoT devices perform only lightweight AES-256-GCM encryption of raw data and generate salted attribute hashes, while fog nodes complete the RLWE-based CP-ABE encapsulation of session keys. This split-path design significantly reduces device-side computational overhead, ensures that neither plaintext nor session keys are revealed to semi-trusted

fog nodes, and naturally integrates with our policy-hiding LSSS construction and epoch/blacklist-based revocation mechanism.

- 2) *Policy-Hiding Attribute Encoding*: We introduce salted hash encoding of attribute labels embedded into LSSS matrices, which conceals the underlying policy even from fog nodes. This prevents inference and profiling attacks while retaining the expressiveness of fine-grained access structures.
- 3) *Forward-Secure, Fine-Grained Revocation*: We propose an efficient revocation mechanism that combines epoch-based key expiration with blacklist verification. This dual strategy enables both user-level and attribute-level revocation while eliminating the costly ciphertext re-encryption process and maintaining forward security.
- 4) *Post-Quantum Authenticity and Integrity*: We integrate CRYSTALS-Dilithium signatures and Poly1305 message authentication codes to guarantee authenticity and integrity in a post-quantum environment, preventing tampering during fog-side delegation and transformation.
- 5) *Comprehensive Performance Evaluation*: We implement the proposed framework in a Raspberry Pi-fog-cloud testbed and evaluate its performance against existing lattice-based CP-ABE baselines. Results demonstrate significant reductions in device-side computation and communication overhead, as well as scalability in revocation management and policy privacy preservation.

The remainder of this paper is organized as follows. Section II reviews related work, and Section III introduces cryptographic preliminaries. Section IV presents our proposed scheme, followed by a formal security analysis in Section V. Section VI provides a detailed evaluation, and Section VII concludes the paper.

II. RELATED WORK

Securing data access in resource-constrained IoT environments requires cryptographic schemes that are not only lightweight and fine-grained but also resilient against emerging quantum threats. This section reviews relevant works across three main domains: (i) classical CP-ABE schemes tailored for IoT, (ii) post-quantum lattice-based CP-ABE schemes, and (iii) supporting techniques for outsourcing, revocation, and authentication.

A. LIGHTWEIGHT AND FOG-ASSISTED CP-ABE FOR IoT

To alleviate the computational load on IoT devices, several classical CP-ABE enhancements have been proposed. Tan et al. [18] introduced a lightweight KP-ABE scheme to minimize pairing operations, making it suitable for constrained devices. Miao et al. [19] and Yin et al. [20] proposed CP-ABE schemes combined with searchable encryption (SE) and fog computing. These designs effectively offload most computational tasks to edge nodes.

Yao et al. [21] enhanced data confidentiality in distributed IoT applications by combining CP-ABE with IOTA's Masked Authenticated Messaging (MAM) protocol. To improve scalability and decentralization, Guo et al. [22] designed a pairing-free multi-authority CP-ABE scheme for edge-cloud environments. For practical deployment, Mendoza-Cardenas et al. [23] implemented CP-ABE over MQTT using Raspberry Pi devices, showcasing feasibility in real-world IoT platforms. However, these schemes rely on classical hardness assumptions and remain vulnerable to quantum adversaries.

B. POST-QUANTUM LATTICE-BASED CP-ABE

To address quantum threats, recent studies have explored lattice-based CP-ABE constructions rooted in the LWE and Ring-LWE problems. Qian and Wu [16] and Sravya et al. [5] proposed Module-LWE-based schemes that emphasize lightweight tree structures and secret key reuse to reduce decryption complexity in IoT. Zhang et al. [6] introduced a novel puncturable Ciphertext-Policy Attribute-Based Encryption (Pt-CP-ABE) scheme founded on the LWE assumption. This scheme allows data owners to specify fine-grained access policies for encrypted data. Additionally, once a ciphertext is decrypted, users can autonomously update their private keys and revoke access to previously accessible ciphertexts by puncturing the associated tags embedded in the ciphertext. Fun and Samsudin [24] presented a Ring-LWE-based CP-ABE leveraging Linear Secret Sharing Schemes (LSSS) with randomized keys to achieve collusion resistance. Yang et al. [25] and Huang et al. [14] developed revocable CP-ABE schemes supporting multi-authority models and attribute-level revocation, but they do not support fog-assisted processing or policy privacy. Yao et al. [2] and Datta et al. [3] introduced decentralized multi-authority CP-ABE schemes under the LWE assumption, enabling authority flexibility and collusion resistance.

Despite these advances, most existing schemes lack support for secure delegation, policy hiding, and efficient ciphertext transformation—capabilities essential for scalable and privacy-preserving access control in fog-assisted IoT settings. In [29], Xu et al. proposed a more efficient and quantum-resistant encryption scheme called Ring-ExpLWE, which replaces the binary error distribution used in Ring-BinLWE with an exponential distribution. To assess its security, they analyzed the scheme's resistance to quantum hybrid attacks by measuring runtime and comparing the standard deviation of the noise polynomial coefficients. Results show that Ring-ExpLWE incurs a higher computational cost for quantum hybrid attacks and exhibits a more dispersed noise distribution than Ring-BinLWE, indicating stronger security under equivalent parameters.

C. REVOCATION, POLICY HIDING, OUTSOURCING, AND AUTHENTICATION ENHANCEMENTS

Several works have explored revocation, policy hiding, outsourcing, and authentication in post-quantum access control.

Revocation Mechanisms: Zhao et al. [17] proposed RL-ABE, a lattice-based scheme supporting attribute revocation through rekeying and ciphertext updates. However, it suffers from high revocation overhead and lacks user-level granularity. Huang et al. [14] designed a revocable ABE resilient to key exposure, though it does not address fog delegation or policy privacy. Wen et al. [15] developed LaRRS for vehicular networks using revocable ring signatures, prioritizing anonymity rather than fine-grained access control. Zhang et al. [6] employed puncturable encryption to enable self-revocation by updating keys after decryption, which is efficient; however, it does not provide centralized enforcement or fog-assisted processing. More recently, Chen et al. [32] presented a revocable attribute-based encryption (RABE-DI) scheme that guarantees both user revocation and verifiable data integrity, enabling users to detect incorrect cloud-side revocations. While efficient within cloud environments, the scheme's reliance on bilinear pairings still incurs high computational cost, making it less suitable for constrained IoT devices.

Policy-Hiding and Updating: Protecting the confidentiality of access policies has been another important research direction. Gu et al. [33] proposed EABE-PUFPH, which achieves full policy hiding and reliable policy updating, ensuring that access structures remain confidential even when policies evolve. Similarly, Li et al. [34] designed PH-MG-ABE, a flexible multi-group policy-hidden ABE scheme that allows dynamic user grouping and hidden access structures, making it suitable for collaborative cloud environments. These works demonstrate the feasibility of maintaining strong confidentiality of access structures, yet they do not integrate efficient revocation mechanisms tailored to IoT-fog environments and lack post-quantum resilience.

Outsourcing Techniques: Bagchi et al. [4] integrated trapdoor sampling and Shamir's secret sharing in a Ring-LWE multi-authority CP-ABE scheme to support partial delegation. They later extended this work [9] for blockchain-based healthcare auditing, but the revocation model remained rigid and imposed heavy key-management costs on users. Xu et al. [29] introduced Ring-ExpLWE to enhance resistance against quantum-hybrid attacks, yet their design lacked revocation support and fog delegation capabilities.

Authentication Enhancements: Post-quantum authentication has also attracted attention. Shen et al. [26] and Wang et al. [27] optimized CRYSTALS-Dilithium for GPU and FPGA platforms, while Lu et al. [28] built lightweight modules for Kyber and Dilithium on embedded systems. Although these works strengthen authentication efficiency, they are not tailored for fine-grained access control and offer no mechanisms for revocation or attribute enforcement.

Despite various advancements, no existing scheme simultaneously supports post-quantum security, fine-grained and privacy-preserving access policies, flexible and user-level revocation, and fog-assisted lightweight delegation tailored for IoT-cloud environments.

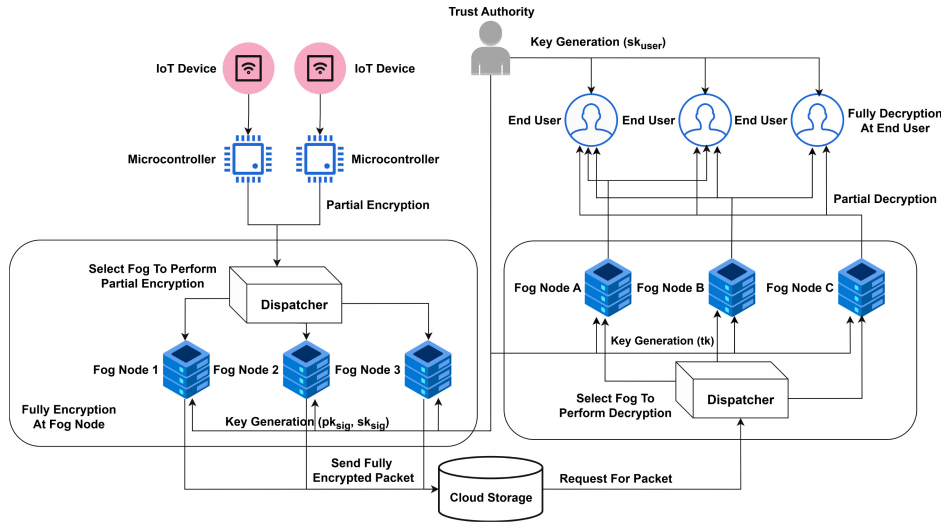


FIGURE 1. Quantum-Resistant Access Control for IoT Cloud System.

III. PRELIMINARIES

This section outlines the fundamental cryptographic tools and assumptions that underpin the proposed scheme, including Ring-LWE hardness, trapdoor sampling via SamplePre, and Linear Secret Sharing Schemes (LSSS) for fine-grained access control.

A. RING LEARNING WITH ERRORS (RING-LWE)

The Ring-LWE assumption is a post-quantum hard problem widely used in lattice-based cryptography. Let $R_q = \mathbb{Z}_q[x]/(f(x))$ be a polynomial ring where $f(x)$ is typically a cyclotomic polynomial.

The Ring-LWE problem states that, given samples of the form $(a(x), b(x) = a(x)s(x) + e(x))$ where $a(x) \in R_q$ is uniformly random, $s(x), e(x) \leftarrow \chi$ are sampled from an error distribution, it is computationally hard to distinguish such samples from uniformly random pairs in $R_q \times R_q$.

The hardness of Ring-LWE has been reduced to worst-case problems on ideal lattices such as GapSVP and SIVP.

In our scheme, Ring-LWE underpins the security of ciphertext-policy ABE operations and session key encapsulation.

B. TRAPDOOR SAMPLING AND SAMPLEPRE ALGORITHM

To securely support lattice-based key generation and policy enforcement, we utilize trapdoor sampling via the SamplePre algorithm introduced by Micciancio and Peikert. This algorithm allows the generation of small-norm solutions to linear equations over a ring, which is crucial for attribute-bound key issuance.

Let $A \in \mathbb{Z}_q^{n \times m}$ be a uniformly random matrix for which a trapdoor basis $T \in \mathbb{Z}^{m \times m}$ is known. Given a syndrome $\mathbf{u} \in \mathbb{Z}_q^n$, the SamplePre algorithm produces a short vector $\mathbf{e} \in \mathbb{Z}^m$ such that:

$$A \cdot \mathbf{e} \equiv \mathbf{u} \pmod{q} \quad \text{and} \quad \|\mathbf{e}\| \leq s,$$

where s is a Gaussian parameter determined by the quality of the trapdoor.

Internally, SamplePre performs discrete Gaussian sampling over lattices using rejection sampling and perturbation vectors derived from the trapdoor basis. This guarantees that the output distribution is statistically close to uniform over the preimage space, thus ensuring both *privacy* and *collusion resistance*.

In our scheme, we use SamplePre to derive ciphertext transformation keys and user secret keys based on hashed attribute labels, ensuring security under the Ring-SIS assumption.

C. LINEAR SECRET SHARING SCHEME (LSSS)

An LSSS is used to express access structures in CP-ABE. Formally, an access policy is encoded as a matrix $M \in \mathbb{Z}_q^{\ell \times t}$, where each row corresponds to an attribute, and a mapping $\rho: [\ell] \rightarrow \mathcal{U}$ associates matrix rows with attributes from the universe $\mathcal{U}$.

A secret value $s \in \mathbb{Z}_q$ is shared by constructing a random vector $\vec{v} = (s, r_2, \dots, r_t)^T$, and shares are computed as $\lambda_i = M_i \cdot \vec{v}$. A user whose attribute set satisfies the policy can reconstruct s via a linear combination of λ_i values. This guarantees that only authorized sets of attributes recover s , while unauthorized sets obtain no useful information.

In our scheme, we enhance LSSS with privacy-preserving hashing. Each attribute label is first hashed with a random salt and then mapped to a row in the LSSS matrix, thereby hiding the access structure while preserving its functionality.

IV. OUR PROPOSED SCHEME

This section introduces the overall system model and provides a detailed explanation of the system components and their interactions. The proposed scheme aims to ensure quantum-resistant, fine-grained, and revocable access control for IoT cloud environments. It achieves this by integrating partial CP-ABE with lattice-based cryptography, fog computing, and proxy re-encryption (PRE). Additionally,

it incorporates policy hiding and attribute revocation to enhance confidentiality and adaptability in dynamic environments.

A. SYSTEM MODEL

Our system model (Fig. 1) comprises seven main entities:

- 1) *Trusted Authority (TA)* is a fully trusted setup entity responsible for system initialization. It generates global public parameters and master secret keys using lattice trapdoor functions to provide post-quantum resilience. The TA issues partial attribute-based secret keys to end users, embedding revocation and policy metadata, and trapdoor keys for fog nodes to support partial decryption and access enforcement.
- 2) *IoT Devices* are microcontroller-based sensors or actuators that collect raw environmental or operational data. Due to limited resources, each IoT device performs lightweight partial encryption, embedding minimal metadata and offloading expensive cryptographic operations to fog nodes.
- 3) *Fog Nodes* are responsible for completing partial encryption, performing delegated decryption, and enforcing revocation policies in coordination with the Trusted Authority. In addition, they offload computationally intensive lattice operations and digital signature verification from IoT devices, ensuring that only lightweight tasks remain at the edge.
- 4) *Dispatcher* is a logically centralized entity that operates in both encryption and decryption phases. It selects suitable fog nodes for completing encryption based on load, proximity, or trust, and routes user decryption requests while enforcing revocation and access policies. Although it does not access plaintext or keys, the Dispatcher plays a critical role in secure coordination, with all sensitive messages (e.g., epoch updates) digitally signed by the TA and verified by fog nodes.
- 5) *IoT Cloud Storage*: A semi-trusted storage entity that stores fully encrypted data packets. While the cloud ensures availability and scalability, it is assumed to be honest-but-curious and cannot decrypt or learn data contents. It may also maintain indexed metadata for packet retrieval.
- 6) *End Users* are authorized users—such as data analysts, system operators, or third-party auditors—who receive partial attribute-based secret keys from the TA. If their attributes satisfy the ciphertext policy and they are not revoked, users can obtain partially decrypted ciphertexts from fog nodes and complete the final decryption locally.

B. PROPOSED CRYPTOGRAPHIC CONSTRUCTION

To enable secure, verifiable, and policy-aware data access in a quantum-resistant IoT cloud environment, our scheme is structured into seven distinct cryptographic phases. Each phase addresses a critical component of the system—from

setup and encryption to decryption and revocation. The design leverages lattice-based cryptography under the Ring-LWE assumption to support fine-grained access control, privacy-preserving policy enforcement, and time-constrained attribute validity.

Table 1 presents the notations and their meaning used in our model.

1) PHASE 1: SYSTEM SETUP

The initial phase of our scheme involves a one-time execution of the Setup algorithm by a trusted authority. This algorithm establishes the cryptographic environment by generating the master public and secret keys that are foundational to all subsequent operations within the system. The security of the entire framework is predicated on the confidentiality of the master secret key.

Step 1: System Initialization

- **Ring Dimension and Modulus Selection:** Choose a ring dimension $n = 256$ and a modulus $q = 2^{13}$ which define the polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$. This balance ensures security and efficient computation.
- **Public Polynomial Generation:** Sample a public polynomial $a \in \mathbb{Z}_q^n$ uniformly at random to be used in all RLWE-based constructions.
- **Master Secret and Public Key Derivation:** Sample secret polynomial s and error polynomial e from a discrete Gaussian distribution $N(0, \sigma^2)$, with $\sigma = 3.2$. Derive the public polynomial

$$p = (a \cdot s) + e \pmod{q}$$

- **Attribute Vector Initialization:** For each attribute in the predefined universe, generate a pair of polynomials (ψ_i, B_i) , where

$$B_i = a \cdot \psi_i + e_i.$$

The secrets ψ_i are stored in the master secret key (MSK), while B_i vectors are published in the public key (MPK). This process enables attribute-based encryption policies.

Step 2: Parameter Finalization

- **Public Parameters Publication:** Publish the public parameter

$$PP = \{a, p, n, q, \sigma, \text{attr_index}, \text{flex_pool}\}$$

which will be used by all system entities for encryption and key generation.

- **Master Secret Key:** Retain the secret polynomial s , attribute secrets ψ_α , and hashing salt in the master key.

$$MSK = \{s, \psi_{\text{table}}\}$$

- **Salt Generation:** Generate a system-wide random salt to hash attribute labels securely into lookup keys. This enables consistent mapping during encryption and decryption.

TABLE 1. Symbol explanation.

Notation	Description
n	Ring dimension
q	Modulus of ring dimension
$\mathbb{Z}_q$	The ring of integers modulo q
R_q	The polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$
σ	Standard deviation for the discrete Gaussian distribution
TA	Trusted Authority, the entity that initializes the system
PP	Public parameters, used for encryption and key generation
MSK	Master Secret Key, the main secret key held by the TA
salt	A system-wide random value used for securely hashing attribute labels
sk_{user}	Secret key for an end user, tied to their attributes and epoch
$pk_{\text{sig}}, sk_{\text{sig}}$	Public/private key pair for the Dilithium post-quantum signature scheme
ψ_{epoch}	Secret polynomial for a given epoch, enabling forward-secure key updates
β, β_i	Vectors of polynomials derived from the master secret for partial decryption
tk	Partial decryption token issued to a fog node
K_{sess}	256-bit symmetric session key (AES-GCM)
U_{attr}	Universe of all possible attributes
H	Cryptographic hash function
L_i	Hashed attribute label $H(a_i \text{salt})$
B_i, ψ_i	Pair of polynomials for attribute i , used in key generation
B_α	Public matrix of all attribute vectors (Master Public Key)
attr_index	Mapping assigning a unique index to each attribute
flex_pool	Set of attributes designated as flexible for fog processing
epoch	Time-period identifier for key validity and revocation
T_{exp}	Expiration timestamp indicating maximum ciphertext validity
current_time	System time used to check against T_{exp} for revocation
AES-GCM	AES in Galois/Counter Mode used for payload encryption/decryption
$u, C_{\text{epoch}}, C_{\text{attr}}$	Values computed during CP-ABE encapsulation (attribute, epoch components)
m	Polynomial encoding of the session key K_{sess}
C_r	Ciphertext component encapsulating the polynomial m
outer _{ct}	Lattice-based ciphertext grouping ($u, C_{\text{epoch}}, C_{\text{attr}}, C_r$)
PolyKey	256-bit symmetric key for the Poly1305 authenticator
PolyTag	Authentication tag generated by Poly1305
CT_{partial}	Partially encrypted ciphertext generated by the IoT device
CT_{final}	Fully encrypted ciphertext produced by the fog node
stripped _{ct}	Ciphertext after the fog node removes flexible-attribute components

- **Attribute Index Mapping:** Assign unique indices to each attribute to facilitate fixed-length vector representations and matrix constructions in the ABE process.

Algorithm 1 System Setup Phase

```

1:  $(n, q, \sigma) \leftarrow \text{SystemParameters}()$ 
2:  $\text{salt} \leftarrow \text{RNG}()$ 
3:  $a \leftarrow \mathcal{R}_q$ 
4:  $s \leftarrow \mathcal{D}_\sigma$ 
5: for each attr  $\in U_{\text{attr}}$  do
6:    $(\psi_{\text{attr}}, B_{\text{attr}}) \leftarrow \text{GenAttributePair}(a, \sigma)$ 
7:    $h_{\text{attr}} \leftarrow H(\text{attr} || \text{salt})$ 
8:    $\text{MSK}.\psi_{\text{table}}[h_{\text{attr}}] \leftarrow \psi_{\text{attr}}$ 
9:    $\text{MPK}.B_{\text{table}}[h_{\text{attr}}] \leftarrow B_{\text{attr}}$ 
10:
11:  $\text{MPK} \leftarrow \{n, q, \sigma, a\}$ 
12:  $\text{MSK} \leftarrow \{s\}$ 
13: return (MPK, MSK, salt)

```

2) PHASE 2: EPOCH-BASED KEY GENERATION

In this phase, the system generates and distributes epoch-specific key materials to both end-users and Fog nodes based on their assigned attributes. This process is managed by a central authority that holds the master secret key. The epoch mechanism allows for periodic key rotation and updates.

Step 1: Attribute Encoding

- **Attribute Labeling with Salt:** Each attribute a_i is transformed into a label $l_i = H(a_i || \text{salt})$, where H is a cryptographic hash function (e.g., SHA-256) and salt is a system-wide secret. This provides unlinkability and protects against attribute inference.

- **Strict vs. Flexible Attributes:** Attributes are separated into:

- **Strict attributes (S):** Mandatory for access and handled at the IoT device.
- **Flexible attributes (F):** Enforced later by the fog node.

Step 2: Key Assignment

- **Epoch Initialization:** First, the system ensures that parameters for the requested epoch exist. If the requested epoch is new, the authority generates a new secret polynomial for the epoch ψ_{epoch} and its corresponding public component B_{epoch} . This new public data is stored for access by the encryption algorithm, while the master keys are updated to reflect the new epoch information. The public epoch component B_{epoch} is saved separately and used by the encryption algorithm to bind ciphertexts to the current epoch. This allows consistent public key rotation without changing existing attribute vectors.

- **End User Secret Key Generation:** The user is issued a secret key containing the secret polynomials (ψ_{vector}) corresponding to their assigned strict attributes. The key also includes the secret polynomial for the current epoch, ψ_{epoch} . This allows the user to perform the final decryption steps.

Algorithm 2 Key Generation Phase

```

1: if any  $H(\text{attr}||\text{salt}) \in \text{Blacklist}$  for  $\text{attr} \in S_{\text{strict}} \cup S_{\text{flex}}$ 
   then abort
2: if epoch  $\notin$  MPK then
3:    $(\psi_{\text{epoch}}, B_{\text{epoch}}) \leftarrow \text{GenAttributePair}(\text{MPK}.a, \sigma)$ 
4:    $\text{UpdateSystemKeys}(\text{MPK}, \text{MSK}, \psi_{\text{epoch}}, B_{\text{epoch}})$ 
6:  $sk_{\text{user}} \leftarrow \{\text{MSK}.\psi_{\text{epoch}}\}$ 
7: for each  $\text{attr} \in S_{\text{strict}}$  do
8:    $h_{\text{attr}} \leftarrow H(\text{attr}||\text{salt})$ 
9:    $sk_{\text{user}} \leftarrow sk_{\text{user}} \cup \{\text{MSK}.\psi_{\text{table}}[h_{\text{attr}}]\}$ 
11:  $tk.\beta \leftarrow \text{InitializeBetaMatrix}()$ 
12: for each  $\text{attr} \in S_{\text{flex}}$  do
13:    $seed_{\text{attr}} \leftarrow H(\text{MSK}.s_{\text{fog\_seed}}||\text{attr})$ 
14:    $\beta_{\text{attr}} \leftarrow \text{PRF}(seed_{\text{attr}})$ 
15:    $tk.\beta[\text{index}_{\text{attr}}] \leftarrow \beta_{\text{attr}}$ 
17:  $(pk_{\text{sig}}, sk_{\text{sig}}) \leftarrow \text{GenSignKeys}()$ 
18: return  $(sk_{\text{user}}, tk, (pk_{\text{sig}}, sk_{\text{sig}}))$ 

```

- *Fog Node Credentials:* There are two key types used by the fog nodes.
 - *Post-Quantum Signing Key:* The fog node generates a Dilithium2 key pair $(pk_{\text{sig}}, sk_{\text{sig}})$ to authenticate ciphertexts.
 - *Partial Decryption Key:* This key contains a matrix of polynomials derived from the master secret. Each polynomial corresponds to one of the user's flexible attributes and is derived using a seeded pseudo-random function (PRF). This allows the fog node to process its portion of the ciphertext without accessing the user's full secret key or the plaintext data.

Algorithm 2 presents the epoch and key generation procedure.

3) PHASE 3: PARTIAL ENCRYPTION IN IOT DEVICE

This phase is performed locally on the IoT device to encrypt raw sensor data prior to transmission. The encryption process is divided into two main components: (1) Symmetric encryption for data confidentiality, and (2) Lattice-based key encapsulation for secure session key delivery. The scheme enforces access control using strict attributes, represented as hashed labels, while deferring evaluation of flexible attributes to fog nodes.

Step 1: Attribute Representation

- The IoT device receives the public parameters PP and its associated strict attribute set S
- Each attribute $a_i \in S$ is hashed with the system-wide salt to form label $l_i = H(a_i||\text{salt})$
- These labels are used to construct the strict attribute matrix $F_s = \text{AttrMatrix}(PP, L_s)$
- *Forwarding Flexible Labels:* The device also hashes and attaches the flexible label set L_f , allowing the fog to enforce the remaining access policies later.

Step 2: Symmetric Encryption

- The IoT device generates a random 256-bit session key $K_{\text{sess}} \leftarrow \text{RNG}(256)$
- The raw data payload M is encrypted using AES-GCM with the session key:

$$(C_{\text{data}}, \text{nonce}) = \text{AES_GCM_Encrypt}(K_{\text{sess}}, M)$$

Step 3: Lattice-Based Key Encapsulation

- A random secret polynomial r is sampled from a Gaussian distribution.
- The session key K_{sess} is encoded into a polynomial m over the ring.

The following public ciphertext components are computed:

$$\begin{aligned}
 u &= a \cdot r + e_u \\
 C_{\text{epoch}} &= B_{\text{epoch}} \cdot r + e_{\text{epoch}} \\
 C_{\text{attr}_i} &= B_i \cdot r + e_i \\
 C_r &= m + e_r
 \end{aligned}$$

Step 4: Integrity and Packaging

- The main lattice components $u, C_{\text{epoch}}, C_{\text{attrs}}, C_r$ are grouped into a single structure called outer_{ct}
- A one-time, 256-bit symmetric key, PolyKey, is generated for message authentication.
- An authentication tag, PolyTag, is computed over the entire serialized structure using PolyKey.
- The device compiles the final CT_{partial} :

$$CT_{\text{partial}} = \text{outer}_{\text{ct}}, C_{\text{data}}, \text{nonce}, T_{\text{exp}}, \text{PolyTag}$$

4) PHASE 4: PARTIAL ENCRYPTION AT FOG NODE

This phase offloads computationally intensive operations to the fog node. The fog node validates the incoming partial ciphertext, completes the encryption process by embedding flexible attribute contributions, and applies a digital signature to ensure data integrity and authenticity.

Step 1: Ciphertext Validation

Upon receiving CT_{partial} , the fog node validates the integrity of the main ciphertext header. It verifies the Poly1305 tag using the corresponding one-time key (PolyKey), which it receives through a separate secure channel. If the tag verification fails, the ciphertext is considered tampered with, and the process is aborted.

Step 2: Flexible Attribute Augmentation

The fog node uses its secret token, a matrix of polynomials denoted as β , to process the flexible attributes.

It computes an aggregate lattice term by summing the products of each flexible attribute ciphertext component (C_i) with its corresponding β_i polynomial. This aggregate term is added to the ciphertext's C_r component, finalizing the key encapsulation to reflect both strict and flexible policies.

Step 3: Digital Signing

The fog node serializes only the modified outer_{ct} header structure. It then signs this serialized data using

its private key (sk_{sig}) and the Dilithium2 post-quantum signature algorithm.

Step 4: Ciphertext Finalization

The fog node outputs the CT_{final} . This is the complete, augmented ciphertext structure containing the original encrypted data, the modified $outer_{ct}$ header, and the newly added fog signature.

5) PHASE 5: PARTIAL DECRYPTION AT FOG NODE

This phase transforms the final ciphertext into a lightweight version for the end-user. The fog node acts as an access enforcer. It first verifies the authenticity of the ciphertext from Phase 4 and then “strips” the flexible-attribute components. This offloads heavy computation from the user’s device.

Step 1: Ciphertext Validation

- Signature Verification: The fog node receives the CT_{final} from the previous phase. It verifies the Dilithium2 digital signature on the ciphertext’s header $outer_{ct}$. This confirms that the ciphertext was authentically processed by a valid fog node in Phase 4. If the signature is invalid, the ciphertext is discarded.

Step 2: Flexible Attribute Stripping

- Using its secret key (β_{matrix}), the fog node calculates the precise mathematical contribution that was added for the flexible attributes during the augmentation step.
- It then subtracts this value from the ciphertext’s C_r component. This stripping action removes the need for the user to process flexible attributes, simplifying the final decryption.

Step 3: Ciphertext Forwarding

- The fog node compiles a new, lightweight $stripped_{ct}$ dictionary containing the simplified ciphertext.
- It computes a new Poly1305 authentication tag over the entire $stripped_{ct}$ structure using this new key.
- The final $stripped_{ct}$ (with its new tag) is then ready to be sent to the end-user. The corresponding PolyKey is sent securely through a separate channel.

6) PHASE 6: END-USER FINAL DECRYPTION PHASE

In this final phase, the end-user receives a refined ciphertext from the fog node and attempts to recover the original plaintext. The process involves verifying integrity, recovering the symmetric session key by reversing the lattice-based encryption, and decrypting the data payload.

Step 1: Integrity Validation

- The end-user receives the $stripped_{ct}$ and its corresponding one-time PolyKey
- The user validates the integrity of the ciphertext by recomputing the Poly1305 MAC over the entire $stripped_{ct}$ structure and comparing it to the received PolyTag. If the tags do not match, the ciphertext is considered tampered with and is discarded.

Step 2: Key Recovery from Lattice Header

- The user applies their secret key (sk_{user}), which contains the secret polynomials (ψ) for their epoch and strict attributes. This recovery is an iterative “noise canceling” process:
 - The calculation starts with the C_r component of the ciphertext.
 - The masking effect of the epoch is removed using the user’s secret epoch key (ψ_{epoch}).
 - For each strict attribute required by the policy, the user applies their secret attribute key (ψ_i) to cancel out that attribute’s masking effect.

- This process implicitly enforces the access policy. If the user is missing any required key, the final result will be incorrect, and decryption will fail. The result of this step is a polynomial that represents the originally encoded session key.

Step 3: AES-Key Reconstruction

- The coefficients of the recovered polynomial S are compared to the modulus q . Coefficients close to or q are decoded as a ‘0’ bit, while coefficients close to $q/2$ are decoded as a ‘1’ bit. This reconstructs the 256-bit AES session key, K_{sess} .

Step 4: Data Decryption

- The user decrypts the AES-GCM ciphertext using the recovered session key:

$$M = \text{AES_GCM_Decrypt}(K_{sess}, C_{data}, \text{nonce})$$

If tag verification succeeds, the plaintext message M is successfully recovered. Otherwise, the operation fails. Finally, the user receives either the plaintext message M , or the symbol $\perp$ if any step fails due to an integrity violation, policy mismatch, or decryption error.

7) PHASE 7: REVOCATION

This phase ensures that revoked or expired credentials cannot be used to access protected IoT data. The scheme implements revocation via two mechanisms: time-based expiration and blacklist verification. These checks are enforced dynamically by the dispatcher and fog nodes at runtime to uphold fine-grained access control and forward security.

Step 1: Expiry-Based Revocation

- Each ciphertext includes a timestamp field T_{exp} , which encodes the maximum validity duration of an attribute or session.
- Fog nodes and end users verify that $\text{current_time} < T_{exp}$
- If the ciphertext is expired, it is immediately rejected—no decryption or processing is attempted.

Step 2: Attribute Blacklist Enforcement

The TA maintains a revocation list of disqualified or expired attribute labels:

$$\text{Revoke}_{\text{Labels}} = H(\text{attr}||\text{salt})$$

where each entry is a hashed representation of a revoked attribute. This list is periodically distributed to fog nodes and dispatchers. If any label in a ciphertext matches an entry in the blacklist, the ciphertext is considered invalid and dropped.

Step 3: Epoch-Based User Revocation

- This mechanism provides broad, user-level revocation. All user keys and ciphertexts are cryptographically tied to a specific epoch number.
- To revoke a group of users, the central authority simply advances the system to a new epoch.
- New, valid secret keys are then issued only to non-revoked users (i.e., users without any revoked attributes).
- Any user who was not re-issued a key for the new epoch will be unable to decrypt any new ciphertexts, effectively revoking their access without needing to add them to a blacklist.

Step 4: Dispatcher Policy Logic

To mitigate key update overhead, our system employs adaptive epoch scheduling, where epoch increments are triggered only upon revocation events rather than at fixed time intervals. This approach avoids unnecessary key regeneration and reduces load during stable periods. Each fog node maintains a revocation epoch cache, synchronized via a lightweight broadcast protocol managed by the Dispatcher. The Dispatcher prevents synchronization issues by including a signed epoch timestamp with each ciphertext or key update. Fog nodes verify epoch freshness using digital signatures and only proceed with transformation if the epoch is current. This ensures revocation coherence without requiring global synchronization or excessive communication overhead. Upon receiving a ciphertext, the dispatcher invokes a revocation check function: $\text{RevokeCheck}(CT_{\text{final}})$:

```
If current_time > CTfinal.Texp
return "REVOKED"
If any H(attr||salt) in blacklist:
return "REVOKED"
return "VALID"
```

This logic provides fast, lightweight enforcement using only timestamp comparisons and hash lookups.

Finally, if the revocation logic returns "VALID", the ciphertext is allowed to proceed through the decryption pipeline. If "REVOKED", the session is terminated, and appropriate logging or alert mechanisms are triggered.

This revocation mechanism supports real-time enforcement with minimal overhead and prevents unauthorized or outdated access across all system layers.

V. SECURITY ANALYSIS

This section formally analyzes the security guarantees of the proposed scheme, with detailed proofs under well-defined cryptographic assumptions. We show that

our system achieves IND-CPA security, policy privacy, ciphertext unforgeability, sound revocation enforcement, forward/backward secrecy, and collusion resistance under the Ring-LWE and EUF-CMA assumptions.

A. IND-CPA SECURITY OF ENCRYPTED SESSION KEYS

Theorem 1 (IND-CPA Security): Under the Ring-LWE assumption, our scheme achieves IND-CPA security for the session key encapsulated via the lattice-based CP-ABE.

Intuition: This means that even if an adversary captures many ciphertexts, they cannot distinguish whether a ciphertext protects one session key or another. In practice, this ensures that the confidentiality of IoT data remains intact under quantum-capable adversaries, since guessing the session key is no better than random chance.

Proof: Consider a game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$:

- 1) *Setup:* $\mathcal{C}$ runs the Setup algorithm to generate public parameters PP , master secret key MSK , and trapdoor T . These are used to generate attribute keys via SamplePre .
- 2) *Query Phase I:* $\mathcal{A}$ adaptively issues attribute key queries for any set S_i such that $S_i \not\subseteq P^*$, where P^* is the challenge policy.
- 3) *Challenge:* $\mathcal{A}$ submits two plaintext messages k_0, k_1 . $\mathcal{C}$ randomly selects a bit $b \leftarrow \{0, 1\}$, and encapsulates k_b under policy P^* :
 - A random vector $\vec{r} \leftarrow \{-1, 0, 1\}^n$ is chosen.
 - Compute $c_1 = A^T r$, and $c_2 = u^T r + \Delta k_b + e$, where $A \in \mathbb{Z}_q^{n \times m}$, $u = As + e'$, and $\Delta = \lfloor q/2 \rfloor$.
 - Output ciphertext $CT^* = (c_1, c_2, \text{Enc}_{\text{ABE}}(P^*))$.
- 4) *Query Phase II:* $\mathcal{A}$ continues querying keys for sets not satisfying P^* .
- 5) *Guess:* $\mathcal{A}$ outputs a guess b' . It wins if $b' = b$.

If $\mathcal{A}$ can distinguish between k_0 and k_1 with non-negligible advantage, we construct a reduction to solve the Ring-LWE problem. Specifically, we embed the LWE challenge $(A, b = As + e)$ into (c_1, c_2) and use $\mathcal{A}$'s success to distinguish whether b is a true LWE sample or uniform. Thus, our scheme is IND-CPA secure.

B. POLICY PRIVACY

Theorem 2 (Policy Hiding): If H is a preimage-resistant hash function and salts are selected randomly, then no PPT adversary can recover attribute names or reconstruct the access policy matrix from the ciphertext.

Intuition: By hashing attribute labels with random salts, the actual access policy (e.g., "doctor AND cardiology department") is hidden. An observer, including semi-trusted fog nodes, only sees random-looking values and cannot learn which attributes are being enforced. This prevents profiling attacks or leakage of sensitive role information.

Proof: Attributes are mapped using salted hashes: $\ell_i = H(a_i || \text{salt}_i)$. Since the salt is chosen uniformly at random per

encryption, and H is preimage-resistant, it is computationally infeasible to invert ℓ_i or link it across ciphertexts.

In the LSSS matrix M , each row is tagged with a hashed attribute ℓ_i , not the raw attribute. As a result, even if the adversary obtains M , it cannot map rows to original attribute labels, preserving policy privacy.

C. UNFORGEABILITY

Theorem 3 (Unforgeability of Ciphertexts): Under the EUF-CMA security of CRYSTALS-Dilithium, no adversary can forge a valid ciphertext-signature pair not previously generated by the signer.

Intuition: No attacker can tamper with or forge a valid ciphertext that would be accepted by the system. For IoT applications, this ensures that malicious nodes cannot inject fake data or impersonate legitimate IoT devices, since any forged ciphertext would fail the Dilithium signature check.

Proof: Let an adversary $\mathcal{A}$ forge a tuple (CT^*, σ^*) such that $\text{Verify}_{\text{Dilithium}}(CT^*, \sigma^*) = 1$ and $CT^* \notin \mathcal{Q}$, where $\mathcal{Q}$ is the set of queries made to the signing oracle.

If $\mathcal{A}$ succeeds, it contradicts the existential unforgeability of Dilithium under chosen-message attacks. Thus, assuming EUF-CMA holds for Dilithium, our ciphertexts are unforgeable.

D. REVOCATION CORRECTNESS AND FORWARD/BACKWARD SECRECY

Theorem 4 (Revocation Soundness): Revoked users cannot decrypt valid ciphertexts, and past ciphertexts remain inaccessible after revocation.

Intuition: Once a user or attribute is revoked, it cannot be used to decrypt any future ciphertexts (forward secrecy), and previously issued keys cannot be used to decrypt older ciphertexts once they expire (backward secrecy). In practice, this means that if an employee leaves an organization, they immediately lose access to both future and past IoT data, even if they keep an old device or key.

Proof: Let $CT = (c_1, c_2, \text{Enc}_{ABE}(P), \text{sig}, T_{exp})$ be a ciphertext encrypted under an access policy P , and let $SK_{u,E}$ denote a user's secret key bound to epoch E . Revocation correctness ensures that decryption succeeds if and only if:

- 1) The user's key epoch E matches the current system epoch E_{sys} .
- 2) The user possesses sufficient attributes to satisfy P .
- 3) The user's attributes are not on the blacklist $\mathcal{B}$.

At decryption time, the fog node or device performs the following checks:

- If $E < E_{sys}$, decryption is rejected (forward secrecy).
- If any $H(a_i || \text{salt}_i) \in \mathcal{B}$, decryption is rejected.
- If the attribute set S does not satisfy P , the LSSS reconstruction fails.

Because epoch keys are not updated for revoked users, and because T_{exp} embedded in the ciphertext ensures that keys from prior epochs cannot be reused (backward secrecy), revoked users are cryptographically prevented from

TABLE 2. Functionality comparison.

Feature	Our Scheme	[7]	[9]	[17]
Post-Quantum Security	Yes	Yes	Yes	Yes
Fine-Grained Access Control	Yes	Yes	Yes	Yes
Policy Hiding	Yes	Yes	No	No
User Revocation	Yes	No	No	No
Forward Security	Yes	No	No	No
Outsourced Encryption/Decryption	Yes	No	No	No
Attribute Revocation	Yes	No	No	Yes

decrypting both future and past ciphertexts. Even if a user retains an older SK , it cannot satisfy the decryption checks due to epoch mismatches and updated blacklists.

Thus, revocation correctness and forward/backward secrecy are enforced by cryptographic coupling of user keys, ciphertext metadata, and system-wide epoch control.

E. COLLUSION RESISTANCE

Theorem 5 (Collusion Resistance): No coalition of users can combine their keys to decrypt a ciphertext unless one of them individually satisfies the policy.

Intuition: Multiple revoked or unauthorized users cannot combine their partial secret keys to break the system. For example, two employees who individually lack sufficient privileges cannot pool their keys to gain unauthorized access to sensitive IoT data. The scheme ensures that only users who independently meet the policy can decrypt.

Proof: Secret keys are generated per attribute using the SamplePre algorithm with embedded salt and user-specific randomness. Since these short vectors are trapdoor-based and linked to disjoint attribute sets, linear combinations across multiple users do not reconstruct a valid decryption vector satisfying the LSSS structure.

Let users $u_1, u_2, \dots, u_k$ hold key shares $\{SK_{u_i}\}$ for attribute sets $\{S_i\}$. If $\cup_i S_i \not\models P$, then due to LSSS linearity and absence of reconstructability, s cannot be recovered. Hence, collusion fails.

VI. EVALUATION

This section evaluates the proposed system. We first compare its functions and computational cost with those of related work. Next, we run experiments to compare the performance of our scheme with related work.

A. FUNCTIONALITY ANALYSIS

Table 2 presents a comparison of the functionalities of our proposed scheme and related works including Yang et al. [7], Bagchi et al. [9], and Zhao et al. [17]. The comparison is done in seven aspects: post-quantum security, fine-grained access control, policy hiding, user revocation, forward security, outsource encryption/decryption, and attribute revocation

The functionality comparison highlights the comprehensive security and access control capabilities of our proposed scheme relative to three existing lattice-based access control frameworks. All schemes offer post-quantum security and fine-grained access control. However, our scheme distinctly supports policy hiding through hashed attributes and ensures user revocation via expiry timestamps and blacklists, whereas [7] and [9] lack effective revocation mechanisms. In contrast, our approach ensures forward security by tying decryption capabilities to valid time-bound credentials. Moreover, our model supports policy updates and outsourced encryption/decryption, optimizing performance for resource-constrained IoT devices—features not fully addressed in the other schemes. Finally, while [17] supports attribute revocation via puncturable encryption, our scheme provides a practical alternative through hashed label blacklisting, combining efficiency and post-quantum resilience.

In addition, our scheme directly addresses policy exposure and privacy risks that are often overlooked in existing lattice-based CP-ABE constructions. In conventional approaches, access policies are embedded in plaintext within the ciphertext, enabling adversaries or fog nodes to infer user attributes and access structures. To mitigate this, we employ a salted attribute hashing mechanism, where each attribute is first hashed together with a global random salt before being mapped into the LSSS matrix. This technique not only hides raw attribute labels but also ensures unlinkability across ciphertexts generated under similar policies. Compared with prior methods that reveal attribute identifiers or rely on limited obfuscation, salted hashing provides a lightweight yet more effective solution to prevent policy leakage and inference attacks without imposing significant computation or communication overhead.

B. COMPUTATION COST COMPARISON

Table 3 summarizes the computational costs of our proposed scheme against existing lattice-based CP-ABE frameworks [7], [9], and [17]. We define the notation used in Table 3 below.

- S_i : The number of attributes owned by a user
- J : The number of attributes in the ciphertext's access policy
- $|S_i \cap J|$: The number of attributes common to both S_i and J
- S' : The set of attributes being revoked
- $|S_i \cap S'|$: The subset of the user's attributes that are revoked
- Exp_{R_P} : An exponentiation operation in the ring R_P
- N_A : The total number of attributes in the global set
- T_{SP} : The cost of one **SamplePre** operation
- N : The total number of attribute authorities in the system
- $|W'|$: The total number of attributes in the data owner's access structure

$|X'_\theta \setminus W'_\theta|$: Attributes held by authority AA_θ that are *not* in W'_θ

RSP: One Ring-Gaussian pre-image sampling operation

N_{strict} : The number of strict attributes assigned to a user

N_{flexible} : The number of flexible attributes assigned to a user

$N_{\text{strict_used}}$: Strict attributes in a ciphertext that a user possesses

$N_{\text{flexible_used}}$: Flexible attributes in a ciphertext that a user possesses

N_{sysuser} : The total number of users in the system

N_{revuser} : The number of users whose keys are revoked

PRF: A pseudorandom-function evaluation to generate a polynomial

S_G : A sampling operation from a Gaussian distribution

CD_{keygen} : Cost of generating a key pair for a digital-signature scheme

Sign: A digital-signature generation operation

Ver: A digital-signature verification operation

T_{decode} : Cost of decoding a polynomial back into an AES key

EpochInit: One-time cost to initialize a new epoch

T_{policy} : Time to check attribute revocation

$\text{Mul}_{R_P}/\text{Add}_{R_P}$: Polynomial multiplication / addition in R_P

Exp_{R_P} : An exponentiation operation in the ring R_P

Compared to existing lattice-based CP-ABE schemes [7], [9], and [17], our proposed system significantly reduces computational overhead on resource-constrained IoT devices by strategically shifting the heavy cryptographic operations to fog nodes. In the key generation phase, the costs for schemes [7] and [9] grow with the number of attributes. This is because they rely on complex sampling and interpolation operations. Our scheme, in contrast, leverages centralized trapdoor-based key generation and efficient pseudorandom function (PRF) evaluations, isolating the device from these intensive tasks. Furthermore, epoch-based key renewal enables forward-secure revocation with minimal recomputation. During encryption, traditional approaches place the full burden on the device, requiring extensive polynomial or ring operations that scale with the access policy size. Our model employs a hybrid encryption structure in which IoT devices handle only lightweight tasks—AES-GCM encryption and attribute hashing—while fog nodes complete the ciphertext construction, including flexible attribute integration, polynomial multiplications, and digital signature generation. This division of labor keeps the device-side encryption cost minimal and nearly constant, regardless of policy complexity.

In the decryption phase, related works such as [7], [9], and [17] require the end-user to process all policy-matching attributes, which makes them impractical for low-power devices. In contrast, our scheme adopts a fog-assisted

TABLE 3. Computation cost comparison.

Scheme	KeyGen	Encryption		Decryption		Revocation
		Device	Fog/Cloud	Device	Fog/Cloud	
[7]	$N_a(2T_{cp} + Mul_v + Add_v) + N_c(N_{ci} + Add_v + Inv)$	$Add_v + N_r Mul_v + (N_a + N_r)(Mul_v + Mul_m + Inv)$	–	$N_c(2Mul_v + Mul_m) + Add_v$	–	–
[9]	$N[h_\theta + Mul_b] + RSP + Add_b$	$ W' (2Mul_b + 2Add_{bR}) + v'_\theta W'_\theta (3Mul_b + 3Add_{bR}) + N(Mul_b + Add_{bR}) + (2Mul_b + 2Add_{bR})$	–	$N W' (Mul_b + Add_{bR}) + S_{uid,\theta} Mul_b + v'_\theta (W'_\theta \cup S_{uid,\theta}) (Mul_b + Add_{bR}) + 2Add_{bR} + N Mul_{lag} + Add_{bR}$	–	–
[17]	$ S_i (H + Inv_{zp} + 5Mul_{zp}) \cdot Mul_R$	$ V (Exp_b + 3Mul_R + 2Add_R) + RSP + 2Mul_R + Add_R$	–	$(S_i \cap J)(Mul_R + Add_R) + (S_i \cap J 2Mul_b) + S_i \cap J Add_R + Sub_R$	–	$(S_i \cap S')(Inv_{zp} + 5Mul_{zp}) Mul_R + V \cap S' (Inv_{zp} Mul_b + Add_R) + (S_i \cap S' 2Add_R)$
Ours	$(N_{strict} + N_{flexible} + 1)H + N_{flexible}PRF + CD_{keygen} + EpochInit$	$AES\text{-}GCM(N_a + 3)S_G + (N_a + 2)Mul_R + (N_a + 3)Add_b + N_a H + MAC_{gen}$	$MAC_{ver} + F Mul_R + Add_R + Sign$	$MAC_{ver} + N_{strict_used} Mul_R + (2N_{strict_used} + 2)Add_R + H + T_{decode} + AES\text{-}GCM$	$Ver + N_{flex_used} Mul_R + Add_R + MAC_{gen}$	$(N_{sysuser} - N_{revuser})T_{keygen} + T_{policy}$

decryption model: the fog node first verifies CRYSTALS-Dilithium signatures and removes the flexible attribute components, then forwards a simplified ciphertext to the user. Consequently, the user's device performs only lightweight tasks—AES-GCM decryption and strict attribute decoding—while the overall decryption cost scales linearly with the number of valid attributes the user holds.

For revocation, scheme [17] provides support but requires cryptographically intensive computations for each revoked user or attribute, while schemes [7] and [9] do not address revocation at all. Our scheme introduces a dual-layer revocation strategy: hashed blacklists are used to invalidate specific attributes, and epoch-based key updates enforce user revocation. Instead of re-encrypting stored data, the trusted authority simply advances the epoch and issues fresh keys to non-revoked users. This approach guarantees forward security and fine-grained access control, while incurring only a modest one-time cost for epoch initialization and key redistribution—significantly lower than the per-user cryptographic overhead in legacy designs.

Overall, our cost model demonstrates that by shifting encryption and revocation burdens to the fog and TA layers, our scheme achieves post-quantum security, fine-grained control, and practical efficiency suitable for scalable fog-assisted IoT deployment.

C. PERFORMANCE ANALYSIS

To evaluate the practical performance of our proposed scheme, we conducted a simulation-based benchmark using real cryptographic operations rather than abstract cost weights. All schemes—our own and those of Yang et al. [7], Bagchi et al. [9], and Zhao et al. [17]—were implemented uniformly to ensure fairness in comparison.

• **Experimental Setup:** Simulations were performed on a workstation with an Intel Core i7-1185G7 CPU @ 3.00 GHz, 16 GB RAM, running Ubuntu 22.04 LTS. This server handled the tasks of the trusted authority and fog nodes, including key generation and encryption/decryption. To emulate realistic edge constraints, we used a Raspberry Pi 4 for device-side operations. All implementations were developed in Python 3.10. The main libraries included NumPy for polynomial and matrix arithmetic, hashlib for cryptographic hashing, cryptography for AES-GCM encryption and Poly1305 MAC, and matplotlib for result visualization. Lattice-based arithmetic (e.g., polynomial multiplication in R_q) was simulated using NumPy arrays with modular reduction to approximate Ring-LWE operations. Trapdoor sampling and digital signing costs were estimated using benchmark results from the official CRYSTALS-Dilithium implementation.

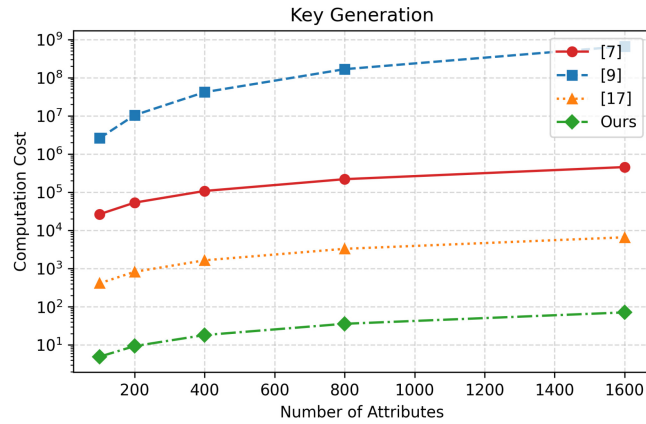


FIGURE 2. Key Generation.

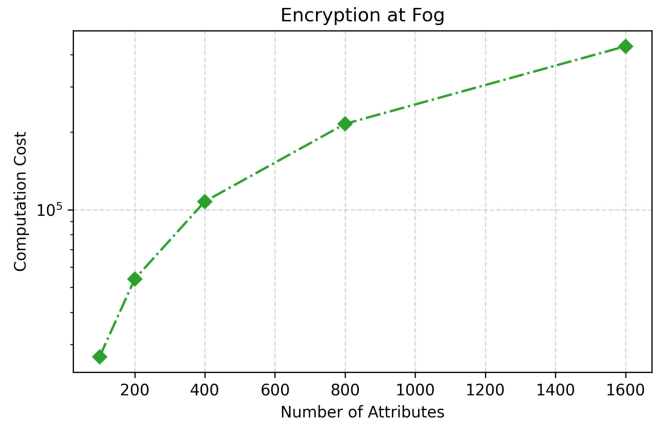


FIGURE 4. Encryption at Fog/Cloud.

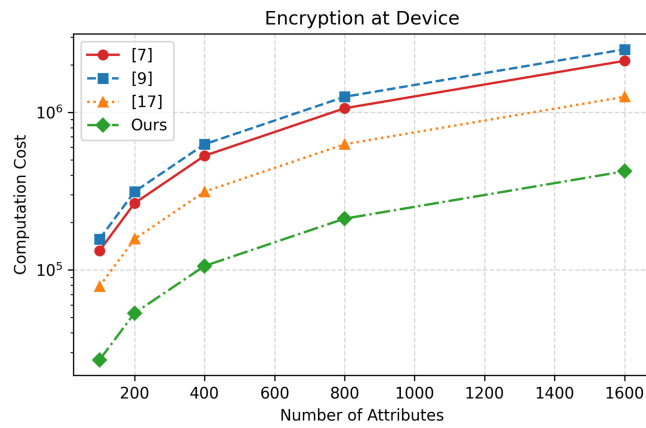


FIGURE 3. Encryption at Device.

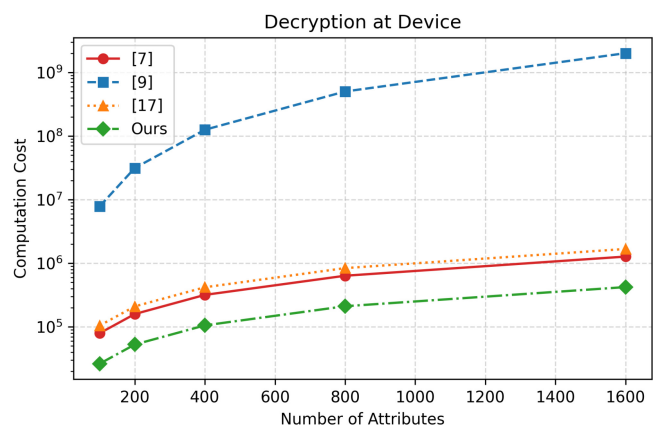


FIGURE 5. Decryption at Device.

Each scheme—our own and the baselines from Yang et al. [7], Bagchi et al. [9], and Zhao et al. [17]—was implemented uniformly for fair comparison. We measured the computation costs for key generation, encryption (device and fog/cloud), decryption, and revocation phases. All reported results were averaged over 10 independent runs to ensure consistency.

Figures 2–6 present the comparative results of key generation, encryption, and decryption across both device and fog/cloud layers, evaluated as a function of the number of attributes.

Figure 2 shows that our scheme achieves the lowest key generation cost and remains nearly constant regardless of attribute count. This is because all heavyweight operations, including trapdoor generation, PRF evaluation, and sampling, are centralized at the TA, leaving IoT devices free of cryptographic load. In contrast, schemes [7] and [9] require attribute-dependent operations such as Shamir sharing and matrix computations on the user side, resulting in linearly increasing costs. Figure 3 highlights a similar advantage in the encryption phase: in our scheme, IoT devices perform only AES-GCM encryption and hashing, while fog nodes handle lattice encapsulation and digital signatures. By comparison, schemes [7] and [9] execute the entire encryption at

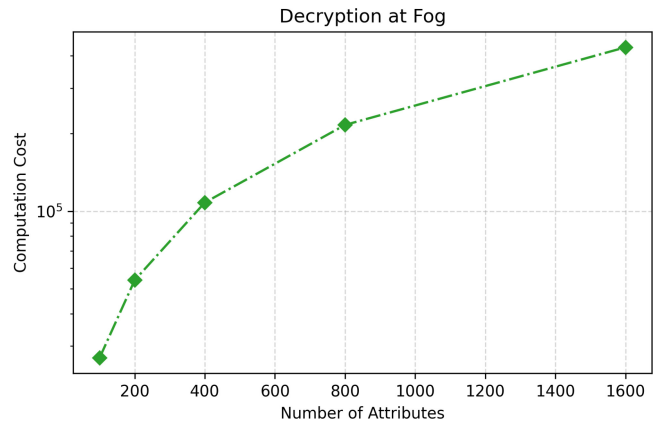


FIGURE 6. Decryption at Fog/Cloud.

the device, causing significantly higher and rapidly growing overhead.

Figure 4 confirms the role of the fog node in handling flexible attribute encapsulation and CRYSTALS-Dilithium signing. The cost at the fog grows linearly with the number of attributes, as expected from its delegated responsibilities. Since [7] and [9] do not support fog delegation, they incur no cost in this phase.

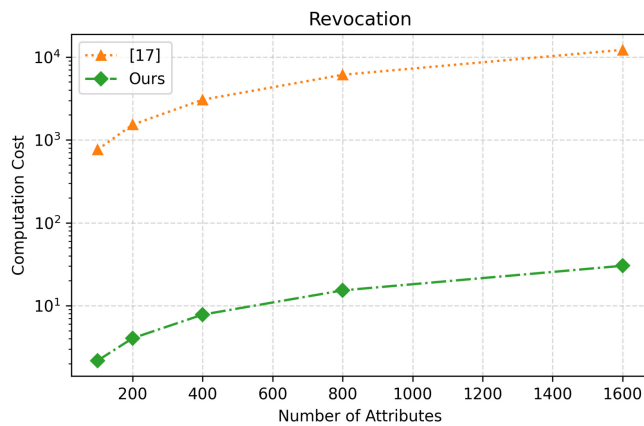


FIGURE 7. Revocation.

Figure 5 shows the decryption cost at the IoT device. In our scheme, this cost remains low and scales gently with the number of attributes. The advantage comes from the fog-assisted design: the fog node processes flexible attributes and verifies CRYSTALS-Dilithium signatures, leaving the device with only lightweight tasks such as AES-GCM decryption and strict attribute checks. In contrast, schemes [7], [9], and [17] require devices to process all matching attributes locally, resulting in higher overhead as policy complexity increases. Figure 6 presents the corresponding fog-node workload in our architecture. The fog performs partial decryption, signature verification, and removal of revoked or unmatched attributes before sending a simplified ciphertext to the user. This cost grows linearly with the number of flexible attributes, demonstrating scalable support for policy-enforced delegation. Baseline schemes do not support fog-layer decryption and therefore incur no cost in this phase.

In addition to core cryptographic operations measurement above, we also measure the performance of attribute revocation between our scheme and scheme [17].

Figure 7 highlights the efficiency of our revocation mechanism. Unlike scheme [17], which incurs significant cryptographic cost due to per-user key updates and ciphertext reprocessing, our scheme maintains a near-constant revocation cost regardless of attribute set size. This performance is achieved through a lightweight revocation check involving hashed blacklist lookups and epoch-based validation—both of which are non-cryptographic and efficiently computable.

VII. CONCLUSION AND FUTURE WORK

In this work, we proposed a fine-grained, lightweight, and quantum-resistant access control framework for IoT-cloud environments. The scheme integrates Ring-LWE-based CP-ABE with a hybrid partial encryption architecture, where IoT devices perform only AES-GCM encryption while fog nodes complete lattice-based encapsulation and policy enforcement. To protect sensitive access structures, we introduced a salted attribute-hashing mechanism embedded into LSSS matrices, ensuring policy privacy even against semi-trusted fog nodes. Furthermore, our epoch-based expiration and

blacklist verification enable efficient user-level and attribute-level revocation with forward secrecy, without requiring ciphertext re-encryption. CRYSTALS-Dilithium signatures and polynomial MACs provide post-quantum authentication and data integrity. Experimental evaluation confirmed that the proposed scheme reduces computational and communication overhead while outperforming state-of-the-art lattice CP-ABE frameworks in efficiency and scalability.

For future work, several promising directions remain. First, we will explore adaptive epoch scheduling that leverages system dynamics and predictive models to further reduce unnecessary key updates while maintaining security. Second, we plan to incorporate lightweight zero-knowledge proofs to allow fog nodes to verifiably demonstrate correct ciphertext transformation without revealing sensitive information. Third, we aim to investigate hardware acceleration of post-quantum primitives such as CRYSTALS-Dilithium and Ring-LWE operations on IoT-friendly platforms, improving real-time performance. Finally, we plan to extend the framework to multi-authority, cross-domain settings with decentralized trust and scalable revocation.

REFERENCES

- [1] R. O'Connor, A. Khalid, M. O'Neill, and W. Liu, "Better security estimates for approximate, IoT-friendly R-LWE cryptosystems," in *Proc. IEEE Asia-Pacific Conf. Circuits Syst. (APCCAS)*, Shenzhen, China, Nov. 2022, pp. 611–615, doi: [10.1109/APCCAS55924.2022.10090405](https://doi.org/10.1109/APCCAS55924.2022.10090405).
- [2] Y.-F. Yao, H.-Y. Chen, Y. Gao, K. Wang, and H.-Y. Yu, "A decentralized multi-authority CP-ABE scheme from LWE," *J. Inf. Security Appl.*, vol. 82, May 2024, Art. no. 103752.
- [3] P. Datta, I. Komargodski, and B. Waters, "Decentralized multi-authority ABE for DNFs from LWE," in *Proc. Adv. Cryptol., Zagreb, Croatia*, 2021, pp. 177–209.
- [4] P. Bagchi, B. Bera, R. Maheshwari, A. K. Das, D. K. Y. Yau, and B. Sikdar, "An efficient and secure post-quantum multi-authority ciphertext-policy attribute-based encryption method using lattice," in *Proc. IEEE INFOCOM Workshops*, Hoboken, NJ, USA, 2023, pp. 1–6, doi: [10.1109/INFOCOMWKSHPS57453.2023.10225757](https://doi.org/10.1109/INFOCOMWKSHPS57453.2023.10225757).
- [5] G. Sravya, P. S. Kumar, and R. Padmavathy, "Secure lattice-based ciphertext-policy attribute-based encryption from module-LWE for cloud storage," in *Proc. IEEE 16th Int. Conf. Cloud Comput. (CLOUD)*, Chicago, IL, USA, 2023, pp. 554–556, doi: [10.1109/CLOUD6044.2023.00074](https://doi.org/10.1109/CLOUD6044.2023.00074).
- [6] T. Zhang, M. Jiang, F. Luo, and Y. Guo, "A lattice-based puncturable CP-ABE scheme with forward security for cloud-assisted IoT," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 26538–26554, Jul. 2025, doi: [10.1109/JIOT.2025.3559567](https://doi.org/10.1109/JIOT.2025.3559567).
- [7] M. Yang, H. Wang, and Z. Wan, "PUL-ABE: An efficient and quantum-resistant CP-ABE with policy update in cloud storage," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 1126–1139, May/Jun. 2024.
- [8] E. Affum, X. Zhang, and X. Wang, "Lattice CP-ABE scheme supporting reduced-OBDD structure," in *Proc. Adv. Comput. Commun. Sci.*, 2021, pp. 131–142.
- [9] P. Bagchi, A. Bisht, A. K. Das, N. Saxena, and M. S. Hossain, "Designing quantum-safe lattice-based multi-authority CP-ABE scheme for blockchain-enabled IoT-based consumer healthcare electronics," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4983–4994, May 2025, doi: [10.1109/TCE.2025.3552021](https://doi.org/10.1109/TCE.2025.3552021).
- [10] X. Fu, Y. Wang, L. You, J. Ning, Z. Hu, and F. Li, "Offline/online lattice-based ciphertext policy attribute-based encryption," *J. Syst. Archit.*, vol. 130, Sep. 2022, Art. no. 102684.

- [11] R. Borhan, M. D. Hossain, Y. Taenaka, and Y. Kadbayashi, "IOSHA—An enhanced hash function for secure and efficient key generation in CRYSTALS-Dilithium for IoT," in *Proc. Int. Conf. Quant. Commun. Netw. Comput. (QCNC)*, Kanazawa, Japan, 2024, pp. 244–252, doi: [10.1109/QCNC62729.2024.00046](https://doi.org/10.1109/QCNC62729.2024.00046).
- [12] S. Ricci, P. Dobias, L. Malina, J. Hajny, and P. Jedlicka, "Hybrid keys in practice: Combining classical, quantum and post-quantum cryptography," *IEEE Access*, vol. 12, pp. 23206–23219, 2024, doi: [10.1109/ACCESS.2024.3364520](https://doi.org/10.1109/ACCESS.2024.3364520).
- [13] O. Popoola, M. A. Rodrigues, J. Marchang, A. Shenfield, A. Ikpehai, and J. Popoola, "An optimized hybrid encryption framework for smart-home healthcare: Ensuring data confidentiality and security," *Internet Things*, vol. 27, Oct. 2024, Art. no. 101314, doi: [10.1016/j.iot.2024.101314](https://doi.org/10.1016/j.iot.2024.101314).
- [14] B. Huang, J. Gao, and X. Li, "Efficient lattice-based revocable attribute-based encryption against decryption-key exposure for cloud file sharing," *J. Cloud Comput.*, vol. 12, p. 37, Mar. 2023, doi: [10.1186/s13677-023-00414-w](https://doi.org/10.1186/s13677-023-00414-w).
- [15] J. Wen, L. Bai, Z. Yang, H. Zhang, H. Wang, and D. He, "LaRRS: Lattice-based revocable ring signature and its application for VANETs," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 739–753, Jan. 2024, doi: [10.1109/TVT.2023.3305037](https://doi.org/10.1109/TVT.2023.3305037).
- [16] X. Qian and W. Wu, "An efficient ciphertext-policy attribute-based encryption scheme from lattices and its implementation," in *Proc. IEEE 6th Int. Conf. Comput. Commun. Syst. (ICCCS)*, Chengdu, China, 2021, pp. 732–742, doi: [10.1109/ICCCS52626.2021.9449182](https://doi.org/10.1109/ICCCS52626.2021.9449182).
- [17] S. Zhao, R. Jiang, and B. Bhargava, "RL-ABE: A revocable lattice attribute-based encryption scheme based on R-LWE problem in cloud storage," *IEEE Trans. Services Comput.*, vol. 15, no. 2, pp. 1026–1035, Mar./Apr. 2022, doi: [10.1109/TSC.2020.2973256](https://doi.org/10.1109/TSC.2020.2973256).
- [18] S. Y. Tan, K. W. Yeow, and S. O. Hwang, "Enhancement of a lightweight attribute-based encryption scheme for the Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6384–6395, Aug. 2019.
- [19] Y. Miao, J. Ma, X. Liu, J. Weng, H. Li, and H. Li, "Lightweight fine-grained search over encrypted data in fog computing," *IEEE Trans. Services Comput.*, vol. 12, no. 5, pp. 772–785, Sep./Oct. 2019, doi: [10.1109/TSC.2018.2823309](https://doi.org/10.1109/TSC.2018.2823309).
- [20] H. Yin, W. Zhang, H. Deng, Z. Qin, and K. Li, "An attribute-based searchable encryption scheme for cloud-assisted IIoT," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 11014–11023, Jun. 2023, doi: [10.1109/JIOT.2023.3242964](https://doi.org/10.1109/JIOT.2023.3242964).
- [21] X. Yao, J. Zhou, X. Du, and S. Zhang, "A CP-ABE and IOTA-based lightweight sensitive-data access-control scheme for IoT," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 40831–40844, Dec. 2024, doi: [10.1109/JIOT.2024.3456553](https://doi.org/10.1109/JIOT.2024.3456553).
- [22] C. Guo, T. Peng, J. Zhang, Y. Yang, Y. Luo, and G. Zhu, "A lightweight pairing-free multi-authority CP-ABE scheme for cloud edge-assisted IoT," in *Proc. 9th Int. Conf. Comput. Commun. Syst. (ICCCS)*, Xi'an, China, 2024, pp. 991–996, doi: [10.1109/ICCCS61882.2024.10602895](https://doi.org/10.1109/ICCCS61882.2024.10602895).
- [23] F. Mendoza-Cardenas, R. S. Leon-Aguilar, and J. L. Quiroz-Arroyo, "CP-ABE encryption over MQTT for an IoT system with Raspberry Pi," in *Proc. 56th Annu. Conf. Inf. Sci. Syst. (CISS)*, Princeton, NJ, USA, 2022, pp. 236–239, doi: [10.1109/CISS53076.2022.9751194](https://doi.org/10.1109/CISS53076.2022.9751194).
- [24] T. S. Fun and A. Samsudin, "Lattice ciphertext-policy attribute-based encryption from ring-LWE," in *Proc. Int. Symp. Technol. Manag. Emerg. Technol. (ISTMET)*, Langkawi, Malaysia, 2015, pp. 258–262, doi: [10.1109/ISTMET.2015.7359040](https://doi.org/10.1109/ISTMET.2015.7359040).
- [25] Y. Yang, J. Sun, Z. Liu, and Y. Qiao, "Practical revocable and multi-authority CP-ABE scheme from RLWE for cloud computing," *J. Inf. Security Appl.*, vol. 65, pp. 103–118, Mar. 2022.
- [26] S. Shen, H. Yang, W. Dai, H. Zhang, Z. Liu, and Y. Zhao, "High-throughput GPU implementation of Dilithium post-quantum digital signature," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 11, pp. 1964–1976, Nov. 2024, doi: [10.1109/TPDS.2024.3453289](https://doi.org/10.1109/TPDS.2024.3453289).
- [27] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient implementation of Dilithium signature scheme on FPGA SoC platform," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 9, pp. 1158–1171, Sep. 2022, doi: [10.1109/TVLSI.2022.3179459](https://doi.org/10.1109/TVLSI.2022.3179459).
- [28] J. Lu, J. Zhang, Z. Luo, A. Li, T. Huang, and D. Liu, "An efficient and configurable hardware architecture of polynomial modular operation for CRYSTALS-Kyber and Dilithium," in *Proc. IEEE Int. Midwest Symp. Circuits Syst. (MWSCAS)*, Springfield, MA, USA, 2024, pp. 29–32, doi: [10.1109/MWSCAS60917.2024.10658892](https://doi.org/10.1109/MWSCAS60917.2024.10658892).
- [29] D. Xu, X. Wang, Y. Hao, Z. Zhang, Q. Hao, and H. Jia, "Ring-ExpLWE: A high-performance and lightweight post-quantum encryption scheme for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 24122–24134, Dec. 2022, doi: [10.1109/JIOT.2022.3189210](https://doi.org/10.1109/JIOT.2022.3189210).
- [30] C. Gentry, C. Peikert, and V. Vaikuntanathan, "Trapdoors for hard lattices and new cryptographic constructions," in *Proc. STOC*, 2008, pp. 197–206.
- [31] M. Yang, H. Wang, and D. He, "Puncturable attribute-based encryption from lattices for classified document sharing," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4028–4042, 2024.
- [32] S. Chen, J. Li, Y. Zhang, and J. Han, "efficient revocable attribute-based encryption with verifiable data integrity," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10441–10451, Mar. 2024, doi: [10.1109/JIOT.2023.3325996](https://doi.org/10.1109/JIOT.2023.3325996).
- [33] C. Gu, J. Li, Y. Zhang, Y. Lu, and J. Shen, "EABE-PUPPH: Efficient attribute-based encryption with reliable policy updating under full policy hiding," *IEEE Trans. Comput.*, early access, Aug. 29, 2025, doi: [10.1109/TC.2025.3603717](https://doi.org/10.1109/TC.2025.3603717).
- [34] J. Li, E. Zhang, J. Han, Y. Zhang, and J. Shen, "PH-MG-ABE: A flexible policy-hidden multigroup attribute-based encryption scheme for secure cloud storage," *IEEE Internet Things J.*, vol. 12, no. 2, pp. 2146–2157, Jan. 2025, doi: [10.1109/JIOT.2024.3468018](https://doi.org/10.1109/JIOT.2024.3468018).



POTCHAKORN POOMEKUM is currently pursuing the Bachelor of Engineering degree in information and communication technology with the School of ICT, Sirindhorn International Institute of Technology, Thammasat University, Thailand. His academic interests lie at the intersection of artificial intelligence and cybersecurity. He is particularly passionate about AI security, cryptography, and data security. He has actively explored how to design secure and privacy-preserving frameworks for modern AI systems, and secure data sharing architectures.



ARISARA SURIYAWONG is currently pursuing the Bachelor of Engineering degree in information and communication technology with the School of ICT, Sirindhorn International Institute of Technology, Thammasat University, Thailand. Her academic interests lie at the intersection of data systems and cybersecurity. She is particularly passionate about cryptography, data security, and system design. She is currently working on data management and application development.



SOMCHART FUGKEAW received the Ph.D. degree in electrical engineering and information systems from The University of Tokyo, Japan. He is an Associate Professor with the School of Information, Computer, and Communication Technology, Sirindhorn International Institute of Technology (SIIT), Thammasat University, Thailand. He is the Head of Cyber Security Lab and a manager of the SIIT Computer and Audiovisual Center. His areas of expertise are in data protection and security, identity and access control management, data privacy-preserving technologies, cloud computing, and blockchain technology.